

CoralGuard

A pre-merge incident prevention agent for the Coral Hackathon

Core idea

CoralGuard answers one high-value engineering question before a pull request is merged:

If we merge this PR today, what is most likely to break in production, who will be affected, and what exact evidence supports that?

It uses Coral to join engineering, observability, support, incident, feature flag, and security data into one evidence-backed risk report.

Why this can win

- It fits the Enterprise Agent track directly.
- It makes Coral essential by using SQL joins across multiple fragmented systems.
- It solves a real problem: preventing production incidents before code ships.
- It is more original than a reactive incident debugger because it works before the incident.
- It can show strong technical depth, strong UX, and clear impact in a short demo.

What the user sees

Input:

```
coralguard assess -pr https://github.com/demo/shop/pull/214
```

Output:

- Merge risk score: low, medium, high, or critical.
- Evidence timeline joining PR changes, failed CI, Sentry errors, metrics, support tickets, Slack incidents, feature flags, and vulnerabilities.
- Blast radius: affected services, customers, routes, owners, and flags.
- Suggested test plan based on similar historical incidents.
- Rollback plan with revert commit, feature flag action, owner, and monitors to watch.
- GitHub-ready PR comment summarizing the risk with evidence.

Coral data sources

- GitHub: PRs, commits, files changed, workflows, issues.
- Sentry: production errors, releases, events.
- Datadog, Grafana, or OpenObserve: metrics, logs, alerts.
- Slack: incident and engineering discussions.
- Linear or Jira: bugs, sprint issues, linked work.

- LaunchDarkly: feature flag rollout and exposure.
- Intercom or HubSpot: customer complaints and account impact.
- OSV: dependency and package vulnerability checks.

Custom Coral source for extra points

Build a custom file-backed source for local CI artifacts:

- `test_failures`
- `coverage_changes`
- `sarif_findings`
- `dependency_diff`
- `codeowners`

This proves learning, strengthens the demo, and targets the source-spec bounty.

Hero demo

Use a seeded demo repo with a realistic risky PR:

- PR changes payment retry logic.
- Sentry shows recent payment timeout errors.
- Intercom shows customer complaints about duplicate charges.
- Datadog shows latency spikes on checkout confirmation.
- Slack has a previous incident thread mentioning the same file path.
- LaunchDarkly shows the new checkout flow is enabled for most users.
- OSV finds a vulnerable dependency touched by the PR.

CoralGuard returns:

High risk. Block merge until checkout retry tests pass, duplicate-charge regression is verified, and the checkout flag is reduced to 10 percent rollout.

MVP build plan

- Simple web dashboard or CLI.
- PR URL input.
- Coral SQL query templates for 5 to 6 sources.
- One custom source for CI/test/security artifacts.
- LLM summary generated only from Coral query results.
- Evidence cards that show the SQL-backed reason for every recommendation.
- Exportable GitHub PR comment.

Submission positioning

CoralGuard turns Coral into a pre-merge reliability immune system: one SQL-powered agent that joins code, incidents, observability, support, flags, and security data to stop production failures before they ship.